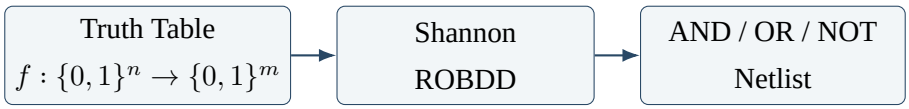
UFUG2106 • Discrete Mathematics

从真值表到逻辑电路

Truth Table to Circuit

数学基础、ROBDD Baseline、代码审计与优化实验



读者对象：已学命题逻辑、基本归纳法与 C++ 基础，但尚未接触逻辑综合、BDD 或 EDA 的本科生。

材料依据：课程 *Final Project (Tentative)*、已发布的 25 个 Stage 1 Cases、Evaluator、本次对照的 C++ 基础版本及独立复现实验。

版本：2026-11-10 ·Learning / Experiment Edition

说明：这是一份教学与实验记录，不是对“全局最优电路”的证明，也不宣称掌握课程未公布的完整评分公式。

目录

阅读导引：先看什么？ **1**

第一章 题目究竟要求构造什么？ **2**

1.1 真值表描述函数，不描述原始电路 2
1.2 目标电路：由信号构成的有向无环图 2
1.3 必须遵守的 Netlist 语言 2
1.4 实际约束与文档不一致处 3

第二章 Area、Delay、Runtime 的数学定义 **4**

2.1 Area 是门成本之和 4
2.2 Delay 是最长的依赖路径 4
2.3 动态 Gate Costs：两个等价结构的符号比较 4
2.4 评分与可行性约束 5

第三章 第一个正确的构造方法：从 DNF 到 Shannon 展开 **6**

3.1 DNF 为什么一定可行？ 6
3.2 限制变量：cofactor 的定义 6
3.3 Shannon Expansion：恒等式与严谨证明 6
3.4 例子：异或函数不需要先认识 XOR 门 7
3.5 递归为什么一定会终止？ 7

第四章 ROBDD：如何把递归式变成可共享的图？ **8**

4.1 决策图的节点语义 8
4.2 两条 Reduction Rules 8
4.3 ROBDD 的 Canonicity 有何前提？ 8
4.4 为什么 bottom-up 构造是可行的？ 9
4.5 不同输出如何共享？ 10
4.6 复杂度与极限 10

第五章 把 ROBDD 映射为允许的 AND2/OR2/NOT 网络 **11**

5.1 MUX 结构与四种常量化简 11
5.2 第二层共享：Structural Hashing 11
5.3 常量传播与简单代数律 12

5.4 为什么最后需要清理无用门? . . . 12
5.5 从两层缓存到最终 Netlist 的完整链条 . . . 12

第六章 对已有 C++ Baseline 的逐模块审计 14
6.1 必须区分：基础算法成立，当前实现还不完善 . . . 14
6.1.1 输入解析与真值表内存布局 . . . 14
6.1.2 BDD Node 与内部 Hash Key . . . 14
6.1.3 Gate Manager：标准化输入并复用 Gate . . . 14
6.1.4 负信号消去：有一个可改进的副作用 . . . 15
6.1.5 多输出根与递归 Gate Mapping . . . 15
6.1.6 构造中的面积记录与实际输出面积 . . . 15
6.1.7 当前代码未处理的事项 . . . 16

第七章 形式化正确性证明与测试方法 17
7.1 综合正确性的四个层次 . . . 17
7.2 独立 Verifier 必须验证哪些性质? . . . 17
7.3 随机测试与差分测试 . . . 18

第八章 已发布 Stage 1 Cases 与 Evaluator：可复现实验 19
8.1 实验事实与评价边界 . . . 19
8.2 Baseline：25/25 PASS 但仅 12/25 满足 Target . . . 19
8.3 试验 A：仅改变 BDD Variable Ordering . . . 20
8.4 试验 B：激进的 De Morgan 面积优先改写 . . . 20
8.5 试验 C：限制局部重写不得恶化 Arrival Time . . . 21
8.6 试验 D：多起点 Variable Ordering Search . . . 21
8.6.1 这些实验不能被误读为一个部署算法 . . . 22
8.7 试验 E：Delay-first 的相反策略（完整 25 Case） . . . 22
8.8 复现指令与结果可信度 . . . 23

第九章 下一步如何优化? 从局部等价式走向全局结构 24
9.1 先定义你希望优化的目标 . . . 24
9.2 方向一：全局变量顺序与 Multi-start 搜索 . . . 24
9.2.1 影响度与支持集 . . . 24
9.2.2 更强的变量排序搜索 . . . 24
9.3 方向二：Polarity-aware / complemented-edge 思想 . . . 25
9.4 方向三：多门重写与 Cut-based Rewriting . . . 25
9.5 方向四：多输出 Sharing 与 Timing Duplication 的冲突 . . . 26
9.6 方向五：Alternative Decomposition，不要锁死 ROBDD . . . 26
9.6.1 DNF/CNF 小函数精确化简 . . . 26
9.6.2 XOR-rich 函数与 Algebraic Normal Form . . . 26
9.6.3 Disjoint-support decomposition . . . 26

9.7 方向六：Critical-path balancing 27
9.8 怎样避免被“局部最优”困住? 27

第十章 实验排查、风险与 Stage 1 报告写法 28
10.1 如何排查具体失败? 28
10.2 每个 Case 应保存的最小数据 28
10.3 复杂度分析的正确口径 28
10.4 如何写 Stage 1 Report? 29
10.5 推荐的两人工作划分 29

第十一章 自测练习与完整解答 30
11.1 题目 30
11.2 解答 30

归纳：从“正确电路”走向“高质量电路” 32

附录 A 原始 C++ Baseline 完整源代码（供逐行研读） 34

附录 B 辅助复现实验的配置记录 36

阅读导引：先看什么?

核心结论

建议按“**问题定义** → **Shannon 恒等式** → **ROBDD** → **门电路生成** → **正确性证明** → **实验优化**”的顺序阅读。**不要先背等价式库**：等价变换优化的是已有的逻辑结构，而 Stage 1 首先需要的是通用的构造算法。

1. **第一次阅读：**第 1–5 章，弄懂每个数学对象、构造动机及其正确性。
2. **第二次阅读：**第 6–8 章，对照 C++ 代码和 Evaluator 数据，理解工程实现。
3. **第三次阅读：**第 9–11 章，将算法优化看成可检验的研究问题，形成自己的实验计划。

全篇严格区分三类信息：**课程规则**（以 PDF 为准）、**实验观测**（基于实际运行）、**研究建议**（需以后续实验检验）。公式中用 a_t 表示门面积参数，用 d_t 表示门延迟参数；二者由每个 Case 单独提供，不是跨 *Case* 固定的物理常数。

第一章　题目究竟要求构造什么?

1.1　真值表描述函数，不描述原始电路

给定 n 个二进制输入和 m 个二进制输出，一份完整的 Truth Table 定义函数

$$f : \{0,1\}^n \longrightarrow \{0,1\}^m, \qquad f(\mathbf{x}) = (f_0(\mathbf{x}), \dots, f_{m-1}(\mathbf{x})). \tag{1.1}$$

输入变量记为 $x_0, \dots, x_{n-1}$，输出变量记为 $y_0, \dots, y_{m-1}$。课程中，第 i 组真值表对应 i 的 n 位二进制表示，x_0 **是最高位**；该组内的第 j 个字符从左至右代表 y_j。

关键认识：函数与电路不同。对任意 x_0, x_1，

$$x_0 \vee x_1 = \neg(\neg x_0 \wedge \neg x_1).$$

两种表达式描述同一个函数，却给出不同的电路。Truth Table 不包含原始电路的历史；我们只能要求新电路与表**功能等价**，无须“反推出原电路”。

1.2　目标电路：由信号构成的有向无环图

一个合法电路 C 由常量 0、原始输入、AND2/OR2/NOT 门及最终输出连线构成。输出可以来自原始输入、常量或者任意先前门的输出；门可有多个使用者，因此图是 DAG（directed acyclic graph），不一定是树。

设输出语义为 $C(\mathbf{x})$，基本的正确性要求为

$$\boxed{\forall \mathbf{x} \in \{0,1\}^n, \quad C(\mathbf{x}) = f(\mathbf{x}).} \tag{1.2}$$

等式对全部 m 个输出同时成立。输入表已经是完整函数说明，因此本项目没有 *don't-care* 输入项，不能把某些行随意视为无关。

1.3　必须遵守的 Netlist 语言

对于 n 个输入，信号 ID 定义为：0 表示常量 0；$1, \dots, n$ 表示 $x_0, \dots, x_{n-1}$；从 $n+1$ 起，每创建一个门按行连续编号。

```
4 OR2 1 2
5 AND2 4 3
4 5
```

当 $n = 3, m = 2$ 时，上例表示 $s_4 = x_0 \vee x_1$、$s_5 = s_4 \wedge x_2$、$y_0 = s_4$、$y_1 = s_5$。末行不是 Gate，而是**输出信号列表**。仅允许向更小的 ID 连线，确保拓扑顺序与无环性质。常量 1 可通过 'NOT 0' 生成。

注意／反例

不要混淆两种编号。构建 BDD 时使用的 Node ID 是内部数据结构索引；最终 Netlist 中的 Signal ID 则必须符合课程规定。两者不是同一个编号空间，不应直接复用。

1.4 实际约束与文档不一致处

课程暂定说明写 $1 \le n \le 18$，但已发布的 Stage 1 中有 $n = 19$ 和 $n = 20$ 的文件。对于 $n = 20$，真值表包含 $2^{20} = 1,048,576$ 个输入赋值。程序应至少能够处理已发布案例，同时向授课方确认最终正式范围。

项目要求**单线程 CPU 程序**，接口为`program input_path output_path`。报告中需要来源清楚、可复现的结果；没有必要实现电路的物理版图或者实际晶体管仿真。

第二章　Area、Delay、Runtime 的数学定义

2.1　Area 是门成本之和

定义门类型集合 $\mathcal{G} = \{\text{AND2}, \text{OR2}, \text{NOT}\}$，第 t 类门的面积成本为 $a_t > 0$，在最终输出 *Netlist* 中出现的个数为 $N_t(C)$。则

$$\boxed{\text{Area}(C) = \sum_{t \in \mathcal{G}} N_t(C) a_t.} \tag{2.1}$$

常量、输入和导线不计面积。若一个门被多个输出共用，它仍只被计算一**次**。另一方面，任何输出都无法到达的“悬空门”即使没有用途，只要仍然列在 Netlist 中，就仍有面积，因此应该清理（dead-gate elimination）。

2.2　Delay 是最长的依赖路径

所有原始输入和常量的到达时刻设为 $T = 0$。按拓扑顺序定义

$$T(g) = \max\{T(u), T(v)\} + d_{\text{AND2}}, \qquad g = \text{AND2}(u, v), \tag{2.2}$$

$$T(g) = \max\{T(u), T(v)\} + d_{\text{OR2}}, \qquad g = \text{OR2}(u, v), \tag{2.3}$$

$$T(g) = T(u) + d_{\text{NOT}}, \qquad g = \text{NOT}(u). \tag{2.4}$$

那么

$$\boxed{\text{Delay}(C) = \max_{0 \le j < m} T(y_j).} \tag{2.5}$$

这里 d_t 是案例给定的抽象传播延迟；相互独立的门并行计算，因此同一层的延迟**不能机械相加**。

命题 2.1 (到达时刻递推的正确性)**.** 在有向无环、门延迟非负的电路中，式 (2.2)–(2.5) 给出的结果，等于从任一输入或常量到任一输出的路径延迟和的最大值。

证明. 按 Gate 的拓扑顺序归纳。基本信号没有 Gate，最长路径权重为 0。若结论对门 g 的全部前驱成立，任何到达 g 的路径必定先到某个直接前驱，再经过 g；故最长长度等于前驱最长长度的最大值加 $d(g)$。对全部输出取最大值得证。 □

2.3　动态 Gate Costs：两个等价结构的符号比较

$$\begin{aligned} f = x_0 \vee x_1 : &\quad A_1 = a_{\text{OR2}}, &&\quad D_1 = d_{\text{OR2}}, \\ f = \neg(\neg x_0 \wedge \neg x_1) : &\quad A_2 = a_{\text{AND2}} + 3a_{\text{NOT}}, &&\quad D_2 = d_{\text{AND2}} + 2d_{\text{NOT}}. \end{aligned}$$

两次输入端 NOT 并行，所以第二个延迟中只需一层输入 NOT，而输出还有一层 NOT。**不能固定认为 $A_2 < A_1$ 或 $D_2 < D_1$**。如果已有部分反相信号可复用，上述 A_2 只是孤立子电路成本；加入到既有 DAG 的增量面积可能更小。

2.4 评分与可行性约束

Evaluator 的 Pass/Fail 首先判断输出格式、功能正确性，以及正值 Runtime/RAM 限制是否满足。超过 Target Delay 仍可能 PASS，但存在惩罚。CSV 提供 `status`、`area`、`delay`、`runtime_ms`、`target_met`；**并未输出最终数值评分公式**。因此不应杜撰“面积占 70%、延迟占 30%”之类的权重。论文中把未达 Target 的案例和满足 Target 的案例严格区分。

第三章　第一个正确的构造方法：从 DNF 到 Shannon 展开

3.1　DNF 为什么一定可行?

对单输出函数 $f:\{0,1\}^n \to \{0,1\}$，令

$$M_{\mathbf{a}}(\mathbf{x}) = \bigwedge_{i=0}^{n-1} \begin{cases} x_i, & a_i = 1, \\ \neg x_i, & a_i = 0. \end{cases}$$

这是仅在 $\mathbf{x} = \mathbf{a}$ 时输出 1 的 minterm，故

$$f(\mathbf{x}) = \bigvee_{\mathbf{a}:f(\mathbf{a})=1} M_{\mathbf{a}}(\mathbf{x}). \tag{3.1}$$

若 On-set 为空，结果为常量 0。这给出非常直接的正确性证明：每个输入只有其本身的 minterm 可能为真。

问题：若 On-set 含 K 行，直接表示约需 $K(n-1)$ 次二输入 AND 以及 $K-1$ 次 OR（不计可共享的负文字），最坏规模呈指数增长。在 $n = 20$ 时并不适合只依赖这种“每行造一个 minterm”的表达。

3.2　限制变量：cofactor 的定义

定义 3.1 (Cofactor)**.** 给定 Boolean Function f 和输入变量 x_i，定义固定该变量后的函数

$$f_{x_i=0}(\mathbf{x}_{\setminus i}) = f(\mathbf{x})\big|_{x_i=0}, \qquad f_{x_i=1}(\mathbf{x}_{\setminus i}) = f(\mathbf{x})\big|_{x_i=1}.$$

其他 $n-1$ 个变量保持自由；两个新函数的定义域都是 $\{0,1\}^{n-1}$。

这里是**把已知输入变量固定成 0 或 1**，并不是“从输出逆推出未知输入”。

3.3　Shannon Expansion：恒等式与严谨证明

定理 3.2 (Shannon Expansion)**.** 对任意 *Boolean Function* f 与其输入变量 x_i，恒有

$$\boxed{f = (\neg x_i \wedge f_{x_i=0}) \vee (x_i \wedge f_{x_i=1}).} \tag{3.2}$$

证明. 分两种互斥且穷尽的赋值情况。若 $x_i = 0$，右式为 $1 \wedge f_{x_i=0} \vee 0 = f_{x_i=0} = f$。若 $x_i = 1$，右式为 $0 \vee 1 \wedge f_{x_i=1} = f_{x_i=1} = f$。两种情况下右式均等于左式，因此是恒等式。 □

这是**精确恒等式**，没有概率假设，没有独立性条件，没有近似误差。只需输入变量取二值。

3.4 例子：异或函数不需要先认识 XOR 门

考虑

x_0	x_1	f
0	0	0
0	1	1
1	0	1
1	1	0

固定 $x_0 = 0$ 得到 $f_0 = x_1$；固定 $x_0 = 1$ 得到 $f_1 = \neg x_1$。所以

$$f = (\neg x_0 \wedge x_1) \vee (x_0 \wedge \neg x_1).$$

这是 XOR 的一个 AND/OR/NOT 实现。没有使用不允许的 XOR Gate。

3.5 递归为什么一定会终止?

每次 Shannon 展开都把一个自由变量固定为常量，剩余自由变量数严格减少；经过最多 n 层到达零变量函数，即常量 0 或 1。通过变量个数的归纳即可推出：任意 *Truth Table* 都可以由这种递归规则正确构造。这说明该方法首先解决的是**可构造性**，还没有解决高质量优化。

核心结论

从 Truth Table 出发的正确思路是“**递归分解已知函数**”，不是“猜测生成表格的隐藏门电路”。DNF 和 Shannon 都能保证正确，但后者更自然地支持**公共子函数共享**。

第四章　ROBDD：如何把递归式变成可共享的图?

4.1　决策图的节点语义

先固定一种变量顺序

$$\pi = (x_{\pi(0)}, x_{\pi(1)}, \dots, x_{\pi(n-1)}),$$

其中 π 为 $\{0, \dots, n-1\}$ 的排列。BDD 有两个终端 $\mathbf{0}, \mathbf{1}$，以及形如 $u = (i, \ell, h)$ 的非终端节点：变量 $x_{\pi(i)}$ 为 0 时访问 ℓ，为 1 时访问 h。

递归定义节点表示的函数 F_u：

$$F_{\mathbf{0}} = 0, \quad F_{\mathbf{1}} = 1, \qquad F_{(i,\ell,h)} = (\neg x_{\pi(i)} \wedge F_\ell) \vee (x_{\pi(i)} \wedge F_h). \tag{4.1}$$

有序要求从根到终端的变量测试顺序与 π 一致，允许跳过某些变量但不允许返回之前的变量。多输出函数对应多个根节点，可复用同一组中间节点。

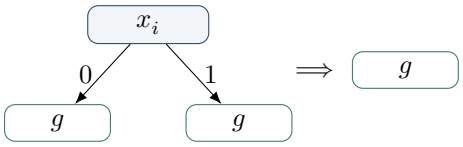
4.2　两条 Reduction Rules

R1. 删除冗余决策： $\ell = h$ 时，直接返回 ℓ。因为 $F_u = F_\ell$，测试 $x_{\pi(i)}$ 毫无意义。

R2. 合并相同节点： 若两个节点同时满足变量位置、低分支、高分支均一致，就保留一个节点：

$$(i, \ell, h) = (i', \ell', h') \quad \Longrightarrow \quad u = u'.$$

用哈希表 `unique[(i,lo,hi)]` 找到已经构造过的节点即可。



当两个分支表示同一子函数，删去决策节点

4.3　ROBDD 的 Canonicity 有何前提?

定理 4.1 (固定顺序的规范性)**.** 在**固定变量顺序** π、固定两种终端、并应用 *R1/R2* 充分约简的前提下，每个 *Boolean Function* 的 *ROBDD* 表示是唯一的（把完全相同的图结构视为同一个图）。

证明思路：按自由变量个数归纳. 零变量时只有两个不同函数 0 和 1，分别由对应终端表示。假设更少变量的所有函数均有唯一约简图。若 f 是常量，则只有终端节点。若不是常量，令 $x_{\pi(i)}$ 是固

定顺序中最靠前、且确实影响 f 的变量：在它之前的变量必须因 R1 被删除，故非终端根只能标记为 i。这个变量的两个 cofactors $f_{x_{\pi(i)}=0}$ 和 $f_{x_{\pi(i)}=1}$ 彼此不同（否则变量不影响 f）；根据归纳假设它们各有唯一的约简子图。根节点因而由唯一三元组 (i, ℓ, h) 确定，R2 禁止另有结构不同但三元组相同的节点。归纳得证。 □

注意／反例

规范性不等于“最小门电路”。该定理只说固定 π 时函数有唯一的 ROBDD；改变变量顺序可能得到不同的图大小和不同的 Netlist。即使某个 ROBDD 节点数很少，直接按 Shannon 展开映射出来的门电路也不一定是 Area 或 Delay 最优的。

4.4 为什么 bottom-up 构造是可行的?

令 $L = 2^n$，对一个输出 y_j，从真值表读出长度 L 的数组：

$$\mathrm{cur}[i] = f_j(\mathrm{bits}_n(i)), \qquad 0 \le i < L.$$

原始顺序 $x_0, x_1, \ldots, x_{n-1}$ 下，每相邻两项仅最后一位不同。因而按 $i = 0, 1, \ldots, L/2 - 1$ 配对

$$(\mathrm{cur}[2i], \mathrm{cur}[2i+1])$$

得到对最后一个变量的两个 cofactors（第一项是 0 分支，第二项是 1 分支）。用 R1/R2 约简并生成父节点，再把结果放进长度 $L/2$ 的 `next` 数组。继续处理倒数第二个变量，直到获得单个根。

Listing 4.1: ROBDD 的 bottom-up 伪代码（位置 v 从后往前）

```
for j in 0..m-1:
    cur = truth_table_column(j)
    for v in n-1 down_to 0:
        next = []
        for k in 0..len(cur)/2-1:
            lo = cur[2*k]
            hi = cur[2*k+1]
            if lo == hi:
                next.push(lo)
            else if unique_table.contains(v,lo,hi):
                next.push(unique_table[v,lo,hi])
            else:
                id = create_BDD_node(v,lo,hi)
                unique_table[v,lo,hi] = id
                next.push(id)
        cur = next
    root[j] = cur[0]
```

数学不变量：在处理完位置 v 之后，数组中的每一项都表示“由前缀变量固定而得到的剩余布尔子函数”，且保持正确的 0/1 分支语义。由配对和 Shannon 恒等式可作归纳证明。

4.5 不同输出如何共享?

对 m 个输出分别遍历对应的 Truth Table 列，但是**使用同一个 Unique Table**。例如

$$y_0 = (x_0 \vee x_1), \qquad y_1 = (x_0 \vee x_1) \wedge x_2,$$

两个函数包含同一个子函数 $x_0 \vee x_1$，共享的 BDD 子图在后续 Gate Mapping 中也有机会形成共享电路。必须理解：**相同 BDD 子节点一定可以共享，但不同 BDD 子节点转换后也可能出现可共享的 Gate**，所以还需要第二层 Structural Hashing。

4.6 复杂度与极限

对每个输出，$L + L/2 + \cdots + 2 < 2L$ 次数组元素处理，若哈希查找期望 $O(1)$，则建立全部输出的 ROBDD 期望运行时间为

$$O(m2^n).$$

这并不意味着 BDD 在所有情况下都很小：**最坏情况下其节点数量仍可能随 n 指数增长**。输入本身已经含 $m2^n$ 个比特/字符，因此读取完整表格就不可避免需要 $\Omega(m2^n)$ 的输入工作量。对于此 baseline，用长度为 $m2^n$ 的表格加上构造缓存，空间为 $O(m2^n + B)$，其中 B 是实际创建的非终端 BDD 节点数。

第五章　把 ROBDD 映射为允许的 AND2/OR2/NOT 网络

5.1　MUX 结构与四种常量化简

对 BDD 节点 (i, ℓ, h)，令 $x = x_{\pi(i)}$、$L = F_\ell$、$H = F_h$。一般映射式为

$$\mathrm{MUX}(x, L, H) = (\neg x \wedge L) \vee (x \wedge H).$$

若已有 $\neg x$ 可复用，通常新建两个 AND2 和一个 OR2；否则还需一个 NOT。baseline 利用常量化简：

条件	精确等价式	门级直觉
$L = H$	$F = L$	直接引用
$L = 0$	$F = x \wedge H$	省掉一支
$H = 0$	$F = \neg x \wedge L$	省掉一支
$L = 1$	$F = \neg x \vee H$	使用 OR
$H = 1$	$F = x \vee L$	使用 OR
其他	$F = (\neg x \wedge L) \vee (x \wedge H)$	一般 MUX

这里每条恒等式都可由 Shannon 恒等式与 $a \wedge 0 = 0$、$a \wedge 1 = a$、$a \vee 1 = 1$、$a \vee 0 = a$ 推出。匹配顺序很重要：若 $L = 0, H = 1$，先用 $L = 0$ 得到 $x \wedge 1 = x$，之后也应进行基本常量折叠，才无需真的创建 AND 门。

5.2　第二层共享：Structural Hashing

BDD Unique Table 以 `(var,low,high)` 为键，对函数决策节点去重。Netlist 的 Gate Cache 以 `(gate-type,fanin0,fanin1)` 为键，对实际门去重，两者不可混同。

对于可交换的 AND2 和 OR2，约定把两个输入 ID 排成升序：

$$(\mathrm{AND2}, a, b) = (\mathrm{AND2}, \min(a, b), \max(a, b)).$$

这样 $a \wedge b$ 与 $b \wedge a$ 命中同一缓存。NOT 只有一个输入，使用 `(NOT,a,0)` 作为键。若缓存命中，返回原 Signal ID，不创建新门；因此额外面积为零。

5.3 常量传播与简单代数律

$$a \wedge 0 = 0, \quad a \wedge 1 = a, \quad a \wedge a = a, \quad a \wedge \neg a = 0,$$
$$a \vee 0 = a, \quad a \vee 1 = 1, \quad a \vee a = a, \quad a \vee \neg a = 1,$$
$$\neg\neg a = a.$$

这些规则可以在创建新 *Gate* 之前执行。要证明工程实现正确，必须注意表达式中的 a 是信号 *ID* 代表的布尔函数，而不是数字常量 a 本身。

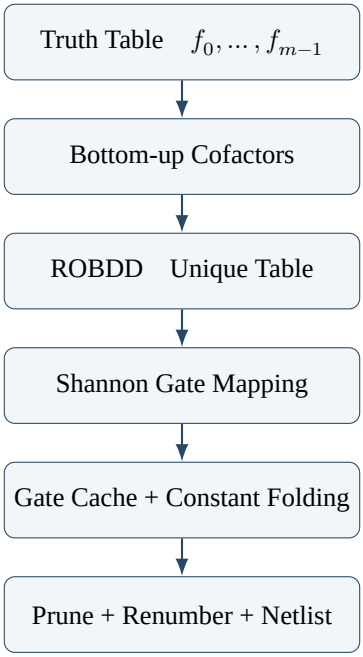
5.4 为什么最后需要清理无用门?

建立电路时，可能创建了中间门，后来由于常量化简、反相消去、缓存重用而没有任何输出引用它。若直接输出所有门，Evaluator 仍会对它们计入 Area。正确做法是从最终输出信号反向沿 Fanin 标记所有可达门，只输出被标记的门，并按照原拓扑顺序为保留门重新分配**连续**的 Signal ID。

命题 5.1 (Dead-gate elimination 保持语义)**.** 删除所有不在任何输出的反向可达集合中的门，并将其余门按拓扑顺序重新编号，不改变任何最终输出的布尔函数。

证明. 未被保留的门没有到任何输出的有向路径，故不能影响输出。保留门之间的依赖边与 Gate 类型不变，重新编号只是对信号名称做一一对应替换；按拓扑顺序归纳，每个保留门的函数不变，最终各输出也不变。 □

5.5 从两层缓存到最终 Netlist 的完整链条



核心结论

这不是“把一堆高级算法随意叠起来”：**Shannon 保障可构造性，ROBDD Reduction 减少重复子函数，Gate Hashing 减少重复物理门，Constant Folding 省掉不必要的运算，Dead-gate Pruning 删除构造过程的废弃物，Netlist Writer 满足课程接口**。每一步都解决不同的确定问题。

第六章　对已有 C++ Baseline 的逐模块审计

6.1　必须区分：基础算法成立，当前实现还不完善

附录 A 嵌入本次依据的原始 `ufug2106_stage1_bdd_baseline.cpp` 源代码。它是教学参考版本，重点在通过所有已发布 Stage 1 的功能检查；它**不是已经完成 Cost-aware Synthesis 的优化器**。

6.1.1　输入解析与真值表内存布局

```
size_t N = 1ull << n;
vector<unsigned char> T(N*m);
for (size_t i=0; i<N; ++i) {
    in >> bits;
    for (int j=0; j<m; ++j) T[i*m+j]=bits[j]-'0';
}
```

内存是“先输入行、后输出列”的 Row-major 布局，$T[i \cdot m + j] = f_j(\text{bits}_n(i))$。各输出构造 BDD 时单独抽取列 j。**审计结论：**代码检查了每个 bit-string 的长度，但未严格验证每个字符都属于 $\{0, 1\}$；工程版应加检查，以避免非法输入转成其他整数。

6.1.2　BDD Node 与内部 Hash Key

```
struct BKey { int v, lo, hi; /* equality */ };
struct BNode { int v, lo, hi; };
if (lo == hi) next[k] = lo;
else if (cache.contains({v,lo,hi})) next[k] = old_id;
else next[k] = create_new_node(v,lo,hi);
```

原代码使用 `unordered_map<BKey,int,H3>`，核心逻辑符合上述两条 BDD reduction rules。节点创建采用从叶向根构造，先出现的子节点一定具有更小的内部索引。

6.1.3　Gate Manager：标准化输入并复用 Gate

```
if (t != 2 && a > b) swap(a,b); // AND/OR commute
GKey key{t,a,b};
if (auto it = gcache.find(key); it != gcache.end())
    return it->second;
int id = n+1+(int)gates.size();
// create gate and cache it
```

其中 $t = 0, 1, 2$ 分别表示 AND2、OR2、NOT。**两类缓存都重要：**BDD Node 相同意味着其代表的函数相同；Gate Key 相同意味着类型与输入连线结构相同。

6.1.4 负信号消去：有一个可改进的副作用

原代码在 `land(a,b)` 与 `lor(a,b)` 中执行类似

```
if (neg(a) == b) return 0;  // or return ONE
```

的问题是，`neg(a)` 不仅查询是否存在补信号，还可能立即创建一个 *NOT Gate*。若实际上 $b \neq \neg a$，这个临时门未必对最终输出有用。后续反向清理会删除其输出中的悬空门，因此**不一定造成最终 Area 的增加**，但会让构造阶段创建更多缓存项、分配更多内存并消耗运行时间。

更合理的接口可分离两种动作：

```
optional<int> lookup_existing_negation(int signal);
int make_negation(int signal); // may create a gate
```

仅在确实需要创建反相器的地方调用后者；互补律检测优先检查现有配对关系，或者用每个信号的逻辑极性结构记录它的互补信号。这属于**确定的工程改进方向**,但其实际速度收益需单独 benchmark。

6.1.5 多输出根与递归 Gate Mapping

```
vector<int> done(nodes.size(), -1);
done[0]=0; done[1]=ONE;
int build(int id) {
    if (done[id] >= 0) return done[id];
    int x = nodes[id].v + 1;
    int lo = build(nodes[id].lo);
    int hi = build(nodes[id].hi);
    // apply simplified Shannon mapping
    return done[id] = mapped_signal;
}
```

`done[id]` 是从 *BDD Node ID* 到 *Netlist Signal ID* 的 Memoization。相同 BDD 子图在多个输出中只映射一次。`x = v+1` 在默认变量顺序 $\pi(i) = i$ 时正确；实验版改变顺序后，必须改成 $x_{\pi(v)}$ 所对应的 Signal ID，即 `perm[v]+1`，否则电路可能悄悄变错。

6.1.6 构造中的面积记录与实际输出面积

源代码中的变量 `A` 会累加所有曾被创建的门的面积；但构造末尾会删除部分门。由此，`A` 不能直接视为最终 Netlist 的 Area（当前程序也没有将它输出）。若以后引入选择候选电路的策略，必须按**保留下来的 Gate 集合**重新统计。`ready` 数组在原始版本里声明了但没有用于 Delay-aware 决策，属于预留设计，不是已经实现的延迟优化。

6.1.7 当前代码未处理的事项

- Gate Costs `aa`，`da`，`ao`，`doo`，`an`，`dn` 均会解析，但**不会用于选择不同逻辑表达式**。
- `target_delay` 被解析但未用于优化；可选 Runtime/RAM 限制没有由综合程序自行作限制性预测，主要交给 Evaluator 检查。
- ROBDD 使用固定顺序 $x_0, x_1, ...$，没有动态变量重排。
- 创建逻辑没有穷举局部等价实现、没有多输出的高级 factoring，也没有全局时序优化。
- 原代码 `n<=22` 检查相较暂定 PDF 的 $n \leq 18$ 更宽松；但并不意味着任意 $n = 22$ 的高密度输入都会有可接受的内存与时间开销。

注意／反例

正确性结论的范围：我们已经使用课程 Evaluator 在已发布 25 个 Stage 1 Cases 上观测到 25/25 PASS。它不能逻辑推出算法在所有合法输入都能满足 Runtime/RAM 限制；功能正确性依赖上文的数学论证，资源可行性必须额外实测。

第七章　形式化正确性证明与测试方法

7.1　综合正确性的四个层次

引理 7.1 (Truth Table 列的解释)**.** 若第 i 个输入赋值按照 x_0 为 *MSB* 的二进制规则排列，并正确提取第 j 列，则数组 `cur[i]` 与 f_j 在第 i 个输入赋值上的取值完全一致。

证明. 由课程输入格式定义和对应的数组下标公式 $i \cdot m + j$ 直接得到。 □

引理 7.2 (Bottom-up BDD 构造)**.** 对固定变量顺序，应用相邻配对、*R1* 删除及 *R2* 合并后，最终第 j 个根节点所表示的函数等于 f_j。

证明. 底层每个元素表示固定全部输入后的常量函数，与原真值表一致。假定某层的子节点正确表示固定前缀、剩余变量上的 cofactors，则下一层构造的父节点由式 (4.1) 准确重组两种取值。R1/R2 仅替换等价节点，不改变函数。沿变量位置反向归纳到根即可。 □

引理 7.3 (BDD Gate Mapping)**.** 若所有 *BDD* 子节点都映射到语义相同的 *Netlist Signal*，则当前父节点通过 *Shannon* 及常量化简得到的信号也表示原节点函数。

证明. 式 (4.1) 与各布尔代数恒等式保证；Gate Cache 只复用运算类型与输入相同的 Gate，不改变函数。 □

定理 7.4 (Baseline 的功能等价)**.** 在输入格式有效、内存分配与运算正常完成的条件下，该 *baseline* 输出的 *Netlist* 对所有 $\mathbf{x} \in \{0,1\}^n$ 满足式 (1.2)，且其 *Gate* 引用构成 *DAG*。

证明. 由真值表列引理、Bottom-up BDD 引理及 Gate Mapping 引理，对各输出根归纳得到功能等价。所有 Gate 在其 Fanin 已存在后才创建，因此 Gate ID 严格大于 Fanin ID；最后的 dead-gate pruning 保持语义与拓扑顺序，重新编号又保持严格较小引用，从而电路无环。 □

7.2　独立 Verifier 必须验证哪些性质?

1. 逐行解析 Gate，检查 Gate 名称是否允许、ID 是否连续、Fanin 是否小于当前 ID、AND/OR 是否恰好两个输入、NOT 是否恰好一个输入。
2. 读取最后一行，确认正好有 m 个已定义 Signal ID。
3. 对所有 2^n 个输入，按 Gate 顺序逐个计算信号值，比较全部 m 个输出。
4. 对 Gate 清单独立重算 Area，并用 Arrival Time 递推独立重算 Delay。

5. 生成专门边界测试：常量 0、常量 1、单一输入直接输出、x_i 的反相、重复输出、XOR、AND、OR、多输出共用同一子函数。

7.3 随机测试与差分测试

在 $n \le 8$ 的小规模上，可随机生成真值表，并用两套独立的生成器（例如 DNF 和 ROBDD）构造电路。对所有输入枚举验证其输出与原表一致。由于两套表示的代码路径不同，这比单纯用同一段构造代码重复比较更能发现错误。随机函数测试不能替代证明，但可以及早发现符号、顺序、编号、常量和缓存错误。

第八章　已发布 Stage 1 Cases 与 Evaluator：可复现实验

8.1　实验事实与评价边界

本次实验使用压缩包中的 25 个 `case_001.txt–case_025.txt`、提供的 Linux Evaluator 和用 `g++ -O2 -std=c++17` 编译的 C++ baseline。以下表中 Area/Delay 对给定 Netlist 是确定性的，Runtime 则受机器、进程启动和系统状态影响，是一次运行的观测值，**不能当作数学常量**。Evaluator 的结果是已发布案例的实测结果，未公开的 Final Tests 仍属于未知分布。

8.2　Baseline：25/25 PASS 但仅 12/25 满足 Target

原始参考版本在已发布案例上全部 PASS；其中有 13 个案例超出 Target Delay。Table 8.1 给出原始记录，**未经过跨机器、跨环境的时间归一化**。

表 8.1: 原始 C++ ROBDD Baseline：25 个案例的实际 Evaluator 记录

Case	n/m	Area	Delay/Target	Runtime (ms)	Target?
001	15/14	135,885	71/77	19.575	Yes
002	14/14	21,835	66/61	14.727	No
003	14/14	28,206	66/64	11.187	No
004	13/10	7,713	57/57	4.002	Yes
005	8/5	1,594	36/36	2.559	Yes
006	12/12	14,272	56/60	3.590	Yes
007	20/15	231,712	89/89	94.723	Yes
008	12/12	15,924	56/62	8.037	Yes
009	12/6	33,212	56/59	6.015	Yes
010	19/8	9,394	75/61	35.418	No
011	10/5	620	47/40	2.868	No
012	10/10	4,157	46/40	3.785	No
013	14/14	11,826	67/70	7.263	Yes
014	20/8	20,808	81/72	58.514	No
015	16/16	38,556	76/81	18.894	Yes
016	10/5	5,008	46/47	2.835	Yes
017	13/12	573	26/25	3.728	No

Case	n/m	Area	Delay/Target	Runtime (ms)	Target?
018	12/12	2,933	41/32	2.728	No
019	18/8	23,692	76/73	17.688	No
020	8/8	824	36/35	2.876	No
021	15/14	69,666	72/72	23.032	Yes
022	8/16	1,286	37/34	3.768	No
023	8/4	262	35/24	2.815	No
024	18/8	180,822	84/83	64.213	No
025	15/15	242,675	71/76	26.026	Yes

8.3 试验 A：仅改变 BDD Variable Ordering

将 $\pi = (x_0, x_1, \dots, x_{n-1})$ 改成反向顺序 $\pi' = (x_{n-1}, \dots, x_0)$；**为保持原函数语义，必须同步重新排列 Truth Table 的索引，并使用正确的原始输入 Signal ID**。其余 Gate Mapping 和缓存逻辑不变。

在这组可重复的变体实验中，natural 与 reverse 都是 25/25 PASS；reverse 的所有案例 *Area* 求和从 $1,103,455$ 变成 $1,084,964$，略有下降，但满足 Target 的案例从 12 个降到 10 个。这里的“Area 求和”只是为了比较本批案例的总资源规模，**不是 Evaluator 最终评分**；单一极大案例也会主导总和。

Case	Natural Area/Delay	Reverse Area/Delay	Target	观察
002	21,835 / 66	17,468 / 66	61	面积明显下降
007	231,712 / 89	219,704 / 96	89	面积下降、延迟失标
017	573 / 26	5,105 / 54	25	反向顺序极不利
023	262 / 35	220 / 35	24	面积下降
024	180,822 / 84	142,370 / 86	83	面积下降、延迟变差

核心结论

变量顺序不是“无关紧要的实现细节”，而是改变 ROBDD 表示复杂度的**全局选择**。反向顺序对 Case 017 的巨大负面影响提醒我们：**不能只记成功案例，更不能把一种固定排列宣布为普适最优**。

8.4 试验 B：激进的 De Morgan 面积优先改写

在原始 Gate Construction 中加入一项启发式：若

$$a_{\text{OR2}} > a_{\text{AND2}} + 3a_{\text{NOT}},$$

则对满足条件的 OR 子结构尝试用 $\neg(\neg u \wedge \neg v)$ 实现；对 AND 有对偶规则。判定只依据孤立子表达式的面积上界，尚未考虑整个 DAG 的延迟和共享关系。最终 Netlist 仍需要 dead-gate cleanup。

实验结果：25/25 PASS；Area 总和由 $1,103,455$ 降至 $799,913$，下降约 27.5%；但满足 Target 的数量从 12/25 降为 3/25。典型的 Case 007 为

$$\underbrace{231,712/89}_{\text{Natural: Area/Delay}} \quad \longrightarrow \quad \underbrace{128,709/103}_{\text{Area-first rewrite}}, \quad D_{\text{target}} = 89.$$

这是一份非常清晰的**面积–延迟冲突证据**。

8.5 试验 C：限制局部重写不得恶化 Arrival Time

进一步比较候选新门的计算时刻：只有在**新实现的输出 Arrival Time 不超过同一组输入下直接 Gate 的 Arrival Time** 时，才接受这个成本更便宜的 De Morgan 改写。例如对 OR 而言，必要的候选比较为

$$T_{\text{direct}} = \max\{T(u), T(v)\} + d_{\text{OR2}},$$
$$T_{\text{DM}} = \max\{T(\neg u), T(\neg v)\} + d_{\text{AND2}} + d_{\text{NOT}}.$$

第二项里的 $T(\neg u)$ 和 $T(\neg v)$ 要使用实际已生成或可复用的反相信号，不能永远认为新增 NOT 的代价与资源不变。

得到 25/25 PASS，12/25 满足 Target；总 Area 为 $1,099,292$，相较 Natural 仅降低约 0.38%，但是没有产生与试验 B 类似的大量延迟失标。

方案	通过数	满足 Target	25 Case 的 Area 总和
Natural ROBDD	25/25	12/25	1,103,455
Reverse ordering	25/25	10/25	1,084,964
Area-first De Morgan	25/25	3/25	799,913
Arrival-time guarded rewrite	25/25	12/25	1,099,292
Delay-first De Morgan	25/25	16/25	1,135,375

注意／反例

试验 C 的“局部安全”并不等同于“已经找到全局 Pareto 最优”。它只限制了所考虑的候选重写相对于当前结构的局部到达时间，既不枚举所有同义结构，也未穷尽多输出共享和未来门的影响。想获得明显的 Area 改善，往往需要跨节点甚至跨输出的重新分解。

8.6 试验 D：多起点 Variable Ordering Search

为了避免仅比较 natural 与 reverse 两个极端，我们针对 8 个较小案例，每个案例用固定种子生成 160 个随机变量排列（`std::mt19937`，seed $= 0, 1, \ldots, 159$），生成候选 Netlist 后以本地独立解析器统计 Area/Delay；挑选的最优候选再交给课程 Evaluator 进行**正确性复核**。这里的最优仅指本次有限候选集合中的最好结果。

Case	Natural Area/Delay	Best sampled Area/Delay	Target	Best seed
004	7,713 / 57	6,429 / 61	57	61
005	1,594 / 36	1,186 / 36	36	21
011	620 / 47	478 / 45	40	47
017	573 / 26	483 / 29	25	147
018	2,933 / 41	1,336 / 36	32	67
020	824 / 36	692 / 36	35	132
022	1,286 / 37	1,008 / 37	34	93
023	262 / 35	158 / 33	24	27

注意 Case 004：以 Area 最优作为唯一准则选出的种子 61，Delay=61，未满足 Target=57；在这 160 个候选里，满足 Target 的一个较优方案为 Area=6,667、Delay=57（seed=113）。这展示了**不能仅保存一个“最小面积候选”**，而应保存一个 Area–Delay Pareto Frontier。

注意 Case 017：本次 160 个随机排列中的最短 Delay 为 27，比原 baseline 的 26 还差；把原始顺序加入候选集合后，目前已知的最好 Delay 是 26，仍超过 Target=25。**多次采样不保证找到可行解**，在此需要研究别的 Gate Mapping 或函数分解技术。

8.6.1 这些实验不能被误读为一个部署算法

本次实验是离线探索，在机器上多次运行综合程序并选结果。这些选择本身需要时间；不能把每次单独综合的 Runtime 当成“160 次搜索的 Runtime”，也不能假定后续隐藏测试允许这么做。若要将 Multi-start 真正纳入最终可执行程序，必须把**所有候选生成和筛选时间**都计入当前 Case 的 Runtime Limit，并且使用程序内部而非 Evaluator 的 Area/Delay 计算完成选择。

8.7 试验 E：Delay-first 的相反策略（完整 25 Case）

我们另外测试了与试验 B 相反的策略：不再要求 De Morgan 改写使面积降低，而是要求候选结构的**计算到达时刻严格早于直接 Gate**。允许以更高 Area 换取更短的 Critical Path。这个启发式依然是局部的，并没有全局求解数学最优电路。

完整测试得到 **25/25 PASS、16/25 满足 Target Delay**，总 Area 为 $1,135,375$，比 Natural 的 $1,103,455$ 略大。下面几个案例十分有解释力：

Case	Natural Area / Delay	Delay-first Area / Delay	Target
010	9,394 / 75	17,804 / 59	61
012	4,157 / 46	7,823 / 37	40
018	2,933 / 41	5,380 / 33	32
023	262 / 35	454 / 27	24
024	180,822 / 84	180,810 / 83	83

其中 Case 010 的 AND2/OR2/NOT 延迟分别为 $1, 4, 1$：对部分 OR 子结构，经过 AND 和 NOT 形成的等价逻辑可能更快，即使面积代价更高。Case 010 的 Delay 从 75 降到 59，由失标变为达标；Case 012 也成功达标。Case 018 和 023 尽管显著缩短了 Delay，仍未达 Target，这说明还需要进一步的结构重组。

核心结论

这一试验比“哪个 Gate 便宜就优先使用哪个”更加重要：**你们实际求解的是多目标综合问题**。面积优先能降低 Area，却可能失去时序；延迟优先能补上某些 Case 的 Target 缺口，却可能显著增加 Area。正确的下一步是维护多个候选，并研究 Pareto Frontier 与共享结构，而不是押注一条永远有效的局部规则。

8.8 复现指令与结果可信度

原始 baseline：

```
g++ -O2 -std=c++17 -o synth ufug2106_stage1_bdd_baseline.cpp
./evaluator ./synth ./stage1 --report baseline.csv
```

变量顺序和门重写的派生实验，使用随文提供的 `variant.cpp`（实验中以环境变量 `ORDER`、`REWRITE` 控制）；随机重启依赖 `randomorder.cpp` 中的 `SEED`。**交付的基本代码附录仅对应原始 baseline，派生变体源码及 CSV 需查看同包实验附件**。对于任何复现，优先核验 **Area/Delay/Pass**，Runtime 应多次重复统计。

第九章　下一步如何优化？从局部等价式走向全局结构

9.1　先定义你希望优化的目标

最重要的步骤不是写更多 Rule，而是声明你的候选选择目标。若课程未给具体 Score，可先采用**两阶段代理目标**：

1. 候选必须**正确、格式合法、程序不超资源限制**。
2. 倾向于满足 $D \le D_{\text{target}}$ 的候选；在这些候选中尽可能降低 Area。若全部不满足 Target，优先研究 Delay 缺口与补救空间，同时保存低 Area 的其他候选。

作为研究工具，可维护 Pareto 集合。若对两个有效电路 C_1, C_2，

$$\text{Area}(C_1) \le \text{Area}(C_2), \qquad \text{Delay}(C_1) \le \text{Delay}(C_2),$$

且至少一个严格小于，则称 C_1 在 Area–Delay 两指标上**支配** C_2。被支配者在这两指标上不存在优势。若把 Runtime 也作为比较指标，必须增加第三维，尤其是在算法生成时间接近上限时。

9.2　方向一：全局变量顺序与 Multi-start 搜索

9.2.1　影响度与支持集

定义布尔函数对变量 x_i 的**布尔差分**：

$$\partial_i f(\mathbf{x}_{\setminus i}) = f(\mathbf{x}_{\setminus i}, 0) \oplus f(\mathbf{x}_{\setminus i}, 1).$$

若该差分恒为零，则函数与 x_i 无关；否则 x_i 属于 f 的 *support*。对多输出，可定义

$$I_i = \sum_{j=0}^{m-1} \sum_{\mathbf{z}\in\{0,1\}^{n-1}} \mathbf{1}\{(f_j)_{x_i=0}(\mathbf{z}) \neq (f_j)_{x_i=1}(\mathbf{z})\}.$$

这个 I_i 是变量的翻转影响度（未归一化）；从完整 Truth Table 可精确计算。它**不能保证**将高影响度变量排前或排后就最优，但可以形成两种有数学动机的 Initial Ordering，再和 natural/reverse/random 比较。

9.2.2　更强的变量排序搜索

- **Adjacent-swap hill climbing**：交换相邻变量，重建 BDD，若 QoR 改善就接受。优点是易实现，缺点是容易陷入局部最优。

- **Sifting：**固定一个变量，依次尝试移动到多个位置，比较结果，再处理其他变量；是常见的 BDD 排序启发式。
- **Multi-start：**从 natural、reverse、支持集启发式和有限随机种子分别起步，不要只相信第一个局部极小值。
- **Budget-aware search：**按 $m2^n$、本地预估 BDD 节点数和 Runtime Limit 决定重建次数，避免对大型 Case 做数百次完整重建。

关键警告：最少的 BDD 节点数未必对应最少的门面积，也未必对应最小 Delay。对目标为 Netlist QoR 的项目，变量排序候选应优先通过**最终映射电路**评估，而不是只根据 BDD Node Count 选择。

9.3 方向二：Polarity-aware / complemented-edge 思想

普通 Shannon Mapping 通常只缓存 f 的正极性表示。可是许多表达式会同时需要 f 与 $\neg f$。如果在内部表示中允许“边标记取反”（complemented edges）或主动缓存正负两种极性的实现，可把大量额外的 NOT 与 De Morgan 转换组织为统一的极性优化问题。

例如 $\neg(a \vee b) = \neg a \wedge \neg b$，真正合算与否依赖：已有反相输入可否复用、其他输出需要何种极性、AND/OR Gate Cost、目前 Critical Path 的 Arrival Time。单独给每个 Gate 套用 De Morgan 只是一阶近似。

一种进阶的动态规划思路是：对某个分解节点分别保存实现 f、实现 $\neg f$ 的多个候选，每个候选记录 (A, D) 或更精确的 Arrival Time Profile；在父节点组合时筛选非支配候选。

注意 / 反例

若不同子节点共享同一个物理 Gate，**简单地把两棵子树的 Area 相加会重复计数**。因此对 DAG 进行成本 DP，不能把共享结构直接当成彼此独立的树；需要结合增量门集合、结构哈希或者在某一阶段接受近似成本，并在最终 Netlist 上复算。

9.4 方向三：多门重写与 Cut-based Rewriting

你的最初设想“Equivalence Library”可升级为：**对小范围输入（例如 3–4 个边界信号），枚举或缓存函数等价的不同 Gate DAG**，而不是单纯列出两侧表达式。

$$a \wedge (b \vee c) \quad \longleftrightarrow \quad (a \wedge b) \vee (a \wedge c)$$

在孤立树中，左边常常省门；但若 $a \wedge b$ 已经在别处存在、或者右边能使关键路径缩短，右边也可能值得考虑。合理的重写框架应执行：

1. 找到一个小的逻辑区域（*cut* 或局部 cone），枚举已知的等价结构；
2. 计算候选的增量 *Area*、输出 Arrival Time 和重构带来的新共享；
3. 在保留功能等价的前提下，接受非支配替代或保留多个候选；
4. 定期清理 dangling nodes，并重新估算全局 Critical Path。

工具 ABC 的公开技术文档把 *balancing*、*rewriting*、*refactoring* 结合成多轮综合流程，这是一个很好的研究方向参照，不是要求你们照搬商业或工业规模工具。[1]

9.5 方向四：多输出 Sharing 与 Timing Duplication 的冲突

共享同一子电路可减少 Area，却可能把很多输出的时序依赖集中到同一位置。一个有意思的折衷是：为某个关键路径**复制一部分逻辑**，以便两条分支采用不同因式分解或者不同深度的实现。

注意理论边界：在本项目的抽象时序模型中，Gate Delay 只由类型决定，Fan-out 本身**没有额外延迟惩罚**；所以单纯复制一个门而保留完全相同的前驱和门类型不会使传播延迟降低，还会增加面积。必须配合**结构重组、并行化、消除关键依赖**，复制才可能有意义。

9.6 方向五：Alternative Decomposition，不要锁死 ROBDD

9.6.1 DNF/CNF 小函数精确化简

当某个局部子函数只依赖少量变量时，可以尝试 K-map、Quine–McCluskey、Espresso 风格的 two-level minimization，再将其映射成二输入门。不要对 $n = 20$ 的整个函数直接运行指数级精确覆盖搜索；应先限制局部支持集规模。

9.6.2 XOR-rich 函数与 Algebraic Normal Form

任意 Boolean Function 都有唯一的 *Algebraic Normal Form*（ANF）：

$$f(x_0, \dots, x_{n-1}) = \bigoplus_{S \subseteq \{0,\dots,n-1\}} c_S \prod_{i \in S} x_i, \quad c_S \in \mathbb{F}_2. \tag{9.1}$$

这里 $\oplus$ 是在 $\mathbb{F}_2$ 上的加法（XOR）。通过布尔立方上的 Möbius Transform 可在 $O(n2^n)$ 的时间内从单输出 Truth Table 求得 ANF 系数。虽然课程没有 XOR Gate，但可用允许门实现

$$a \oplus b = (a \wedge \neg b) \vee (\neg a \wedge b).$$

如果某些测试函数的 ANF 项极少（如奇偶校验），则从 ANF 生成的表达式可能比普通 Shannon 展开更有竞争力；相反，一般函数可能具有非常密集的 ANF，不应盲目采用。

9.6.3 Disjoint-support decomposition

如果 $f(\mathbf{x}) = g(h(\mathbf{u}), \mathbf{v})$ 且 $\mathbf{u}, \mathbf{v}$ 不相交，可以把复杂函数拆成依赖较少输入的子函数。你可以先用布尔差分检测变量的实际 support，再探索变量组是否存在可分离的结构。这一方向比单个 Gate 的简单等价式更加**全局**。

[1] ABC Logic Synthesis and Verification Group 文档：https://people.eecs.berkeley.edu/~alanmi/abc_old/。

9.7 方向六：Critical-path balancing

二输入 AND/OR 的结合律保证

$$(a \wedge b) \wedge c = a \wedge (b \wedge c), \qquad (a \vee b) \vee c = a \vee (b \vee c).$$

若输入 Arrival Time 不同，构造**尽量平衡的运算树**有机会缩短最长依赖链。例如 4 个输入做 AND，用连续左结合需要 3 层 AND，而两两并行后合并只需 2 层。

但真实的共享 DAG 不一定允许自由重新组合：改写可能破坏其他输出复用的已有中间结果，使 Area 增长。建议先对纯 *AND/OR*、引用数为 *1* 的局部树实验 balancing；再进一步考虑带共享的复杂 cone。

9.8 怎样避免被“局部最优”困住?

1. **建立候选池而非单一当前最好电路：**存储若干具有不同 Area/Delay trade-off 的 Pareto 候选。
2. **变量顺序与局部门重写交替迭代：**一次变量重排可开启新的共享机会，一次 Gate factoring 又可能改变下一轮重排评价。
3. **允许有限的中性重写：**部分当前不减少 Area 的等价变换可打开下一轮公共子表达式机会，须有轮次/资源上界与去重机制。
4. **跨表达方式搜索：**ROBDD、ANF、局部 DNF、AIG（AND-inverter graph）表达的搜索空间不同，不要只选一条路线。
5. **根据失败类型分类定位：**Area 很大、Delay 失标、Runtime 超限是三个不同问题，避免对不相干的瓶颈做优化。
6. **保留可恢复的基准：**任何新方法都不得删除上一轮功能正确、表现稳定的版本；用编译选项或配置切换，做逐案对照。

第十章　实验排查、风险与 Stage 1 报告写法

10.1　如何排查具体失败?

现象	优先检查
FAIL / 格式错误	ID 是否连续；Fanin 是否较小；NOT 参数个数；末行是否含恰好 m 项；是否引用了清理掉的 Gate。
FAIL / 功能错误	Truth Table 最高位方向；BDD 0/1 分支；变量重排后的索引映射；多输出 Root 映射；NOT/AND/OR 常量规则。
FAIL / 资源限制	输入大小、临时反相器数量、hash reserve、变量顺序搜索次数、是否重复构造大规模 BDD。
PASS / Area 巨大	BDD Node Count、不同输出的共享率、无用门清理、重复 NOT、AND/OR 非对称成本、局部 factoring 机会。
PASS / Delay 超标	Critical Path 逐门回溯、到达时刻、变量分解层次、De Morgan 替换、可并行化的 AND/OR 子树。
PASS / 时延不稳定	多次重复、机器负载、I/O、进程启动，区分 Runtime 和 Circuit Delay。

10.2　每个 Case 应保存的最小数据

建议构建按 Case 一行的 CSV，至少记录下列字段:

```
case,n,m,gate_costs,target_delay,method,order,seed,valid,
area,delay,delay_excess,runtime_ms,bdd_nodes,
emitted_gates,reused_gates
```

其中

$$\text{delay_excess} = \max\{0, \text{Delay}(C) - D_{\text{target}}\}.$$

对同一版本反复测 Runtime，分别记录中位数与离散程度；对不同版本保持相同的测试机器、编译优化标志和 Evaluator 设定。**不要仅报告平均 Area 变化**：同时报告逐案胜负、Target 达成数，以及明显退化的最坏案例。

10.3　复杂度分析的正确口径

若输入本身包含 $m2^n$ 位，**读取成本至少为** $\Omega(m2^n)$，这是无可避免的。因此“算法时间是指数的”并不自动意味着实现很糟；需要比较是否超出输入读取规模太多，或者是否发生新的搜索爆炸。

这份 baseline 的主构造阶段是 $O(m2^n)$ 的期望哈希工作量，再加上 $O(B)$ 的 Gate Mapping 和按 Netlist 规模的输出。若在全表上做每个变量的影响度扫描，大致增加 $O(nm2^n)$ 的预处理；若重新构造 R 次 BDD，则粗略上界又会乘上 R。必须在正式 Runtime Limit 下评估这类改进是否值得。

10.4 如何写 Stage 1 Report?

根据课程说明, Stage 1 要提交源码和报告, 报告至少应包括 Method 与每个测试案例的 Pass/Fail、Area、Delay、Runtime。推荐结构:

1. **Problem Definition：**写清数学对象、Netlist 格式、Area/Delay 定义和独立约束。
2. **Baseline Method：**Shannon Expansion、ROBDD 两种 Reduction、共享缓存、Gate Mapping、无用门清理；给出伪代码与复杂度。
3. **Correctness：**简要归纳证明，辅以独立 Verifier 和边界测试描述。
4. **Experimental Setup：**编译器版本、编译参数、CPU 环境、Evaluator 命令、运行次数与 Runtime 单位。
5. **Per-case Results：**完整 25 案例表；注明未满足 Target 的案例虽然可能 PASS，评分效果仍未知。
6. **Failure Analysis / Future Work：**依据实际案例划分瓶颈；给出可验证的 Stage 2 改进方向。

10.5 推荐的两人工作划分

- A 同学主要负责 Boolean Decomposition、ROBDD 的数学正确性、变量重排和 Alternative Mapping。
- B 同学主要负责 Gate Manager、独立 Verifier、性能测量、实验脚本和 Report 的结果部分。
- 两人共同审查数据定义、代表性案例和最终关键代码；因为 Live Q&A 要求双方都能解释方法及实验结论。

Stage 2 最重要的加分机会，不只是“结果变好”，而是能解释为什么修改有效、何时无效，以及如何通过统一的实验框架识别这一点。

第十一章　自测练习与完整解答

11.1　题目

练习 1. 给定 $f(x_0, x_1, x_2) = (x_0 \vee x_1) \wedge x_2$。写出对 x_2 的两个 cofactors 及其 Shannon Expansion；说明哪个变量顺序可能使这个分解非常简单。

练习 2. 给定 $f(x_0, x_1, x_2) = x_1$。为什么在测试 x_0 的 BDD 节点上必有 $\ell = h$？哪条 Reduction Rule 生效?

练习 3. 给定门代价 $(a_{\text{AND}}, d_{\text{AND}}) = (2, 2)$、$(a_{\text{OR}}, d_{\text{OR}}) = (8, 3)$、$(a_{\text{NOT}}, d_{\text{NOT}}) = (1, 1)$。对 $f = x_0 \vee x_1$ 比较直接 OR 与 De Morgan 表达的 Area/Delay；当目标 Delay 为 3 时，应该如何描述风险?

练习 4. 判断下述断言是否正确，并说明理由：“减少 *BDD Node Count* 一定减少最终 *Area*；对某个 *Gate* 使用更便宜的等价式一定使最终 *Circuit Area* 下降。”

练习 5. 给 $n = 3, m = 2$ 的 Netlist：`4 OR2 1 2`、`5 AND2 4 3`，末行 `4 5`。列出 $(x_0, x_1, x_2) = (0, 1, 1)$ 时各门值，说明 Gate 4 是否在 Area 中重复计数。

练习 6. 为什么在 $D_{\text{target}} = 25$ 的 Case 017 中，无法从随机 160 种排列的失标结果推断“目标一定不可能实现”?

11.2　解答

解 1. $f_{x_2=0} = 0$，$f_{x_2=1} = x_0 \vee x_1$，所以 $f = (\neg x_2 \wedge 0) \vee (x_2 \wedge (x_0 \vee x_1)) = x_2 \wedge (x_0 \vee x_1)$。若优先按 x_2 分解，该节点的低分支直接是常量 0，映射较简单；这不构成对全部函数的最优变量顺序保证。

解 2. 固定 $x_0 = 0$ 和 $x_0 = 1$ 都得到同一个函数 x_1，故低、高分支函数一致，R1 消除无关决策节点。

解 3. 直接 OR：$A = 8, D = 3$；De Morgan：$A = 2 + 3 = 5, D = 2 + 2 = 4$。后一实现面积小但 Delay 超过 Target=3；依课程规则这不导致功能检查失败，但可能受到 Delay 惩罚。

解 4. 均不正确。ROBDD 节点数量与不同 Gate 类型、极性复用、共享程度及 Delay 映射有关，不能保证单调；局部更便宜的改写可能增加 NOT、破坏共享或需要别的结构，最终面积必须以保留 Gate 集合重新核算。

解 5. $s_4 = 0 \vee 1 = 1$；$s_5 = s_4 \wedge 1 = 1$；因此 $(y_0, y_1) = (1, 1)$。Gate 4 被两处使用仍是一个物理门，面积只计一次。

解 6. 因为仅检验了有限排列，尚未检验所有 $n!$ 种顺序、不同的 Gate Mapping、更深层的逻辑重写和新的函数分解。已知结果只证明该候选集合中未发现满足目标的方案，不能证明不存在。

归纳：从“正确电路”走向“高质量电路”

核心结论

Stage 1 的核心胜利是可证明地构造正确电路： Truth Table → Cofactor → Shannon → ROBDD → Netlist。

真正的挑战从这里开始： 动态成本、变量顺序、正负极性、共享和 Critical Path 相互影响。已完成实验清楚表明：只做 De Morgan 面积压缩会损害 Delay；只改变量顺序则可能大好也可能大坏。

接下来建议： 先改进副作用与计量基础，做 Arrival-time-aware Gate Manager；再在有限、可验证的 *Runtime* 预算内加入 Variable Ordering 搜索、多候选 Pareto 保留、局部 Cut Rewriting，并逐案分析效果。不要把某一个规则或某一种表示法误认成全局最优算法。

参考文献

[1] UFUG2106 Course Team. *Final Project: Truth Table to Circuit (Tentative)*, pp. 1–4; Stage 1 benchmark cases and provided evaluator, as supplied to the students, November 2026.

[2] Oscar Levin. *Discrete Mathematics: An Open Introduction*, 4th edition, 2025. Chapter 1 (Propositional Logic) and Chapter 2 (Discrete Graph Structures). https://discrete.openmathbooks.org/. 注：该教材可用于复习命题逻辑，不应把本文的 ROBDD/综合实验归因于教材。

[3] R. E. Bryant. “Graph-Based Algorithms for Boolean Function Manipulation.” *IEEE Transactions on Computers* 35(8):677–691, 1986. DOI: https://doi.org/10.1109/TC.1986.1676819. (ROBDD 与固定变量顺序的经典来源。)

[4] S. J. Friedman and K. J. Supowit. *Finding the Optimal Variable Ordering for Binary Decision Diagrams*. Princeton Technical Report TR-058-86, 1986. https://www.cs.princeton.edu/research/techreps/586. (变量顺序影响 BDD 规模。)

[5] Berkeley Logic Synthesis and Verification Group. *ABC: A System for Sequential Synthesis and Verification*. https://people.eecs.berkeley.edu/~alanmi/abc_old/. (AIG balancing、rewriting、refactoring 的方法参考。)

附录 A　原始 C++ Baseline 完整源代码（供逐行研读）

下面嵌入的是本次实验依据的**原始教学参考版本**。它已通过已发布 25 个 Stage 1 Cases，但尚未优化变量排序或使用动态 Gate Costs 决策，详见第 6 章。代码由课程之外的实验流程生成，不代表教师指定算法。**编译时仍须以附件中的独立.cpp 文件为准**。

Listing A.1: C++17 ROBDD baseline 完整源码

```
#include <bits/stdc++.h>
using namespace std;
struct BKey {int v,lo,hi;bool operator==(const BKey&o)const{return v==o.v && lo==o.lo && hi==o.hi;}};
struct H3 {size_t operator()(const BKey&k)const { uint64_t z=((uint64_t)(unsigned)k.v<<55)^((uint64_t)(
    unsigned)k.lo<<26)^(unsigned)k.hi; z^=z>>30;z*=0xbf58476d1ce4e5b9ULL;z^=z>>27;z*=0x94d049bb133111ebULL;
    return z^(z>>31);}};
struct BNode{int v,lo,hi;};
struct Gate{string type;int a,b;};
struct GKey{int t,a,b;bool operator==(const GKey&o)const{return t==o.t&&a==o.a&&b==o.b;}};
struct HKey{size_t operator()(const GKey &k)const{return H3{}({k.t,k.a,k.b});}};
int main(int argc,char**argv){
 if(argc<3)return 2;
 ifstream in(argv[1]);ofstream out(argv[2]);if(!in||!out)return 3;
 int n,m;in>>n>>m; if(n<1 || n>22 || m<1)return 4;
 size_t N=1ull<<n; vector<unsigned char> T(N*m);string bits;
 for(size_t i=0;i<N;i++){in>>bits; if((int)bits.size()!=m)return 5;for(int j=0;j<m;j++)T[i*m+j]=bits[j]-'0'
     ;}
 int aa,da,ao,doo,an,dn,D;in>>aa>>da>>ao>>doo>>an>>dn>>D;
 // BDD global canonical graph, terminal 0 and 1.
 vector<BNode> nodes;nodes.reserve(N);nodes.push_back({-1,0,0}); nodes.push_back({-1,1,1});
 unordered_map<BKey,int,H3> cache;cache.reserve(min((size_t)N*2,(size_t)5000000));
 vector<int> roots(m);vector<int> cur(N),next;
 for(int j=0;j<m;j++){
  cur.resize(N);
  for(size_t i=0;i<N;i++)cur[i]=T[i*m+j];
  size_t count=N;
  for(int v=n-1;v>=0;v--){
   next.resize(count/2);
   for(size_t k=0;k<count/2;k++){
    int lo=cur[2*k],hi=cur[2*k+1];
    if(lo==hi){next[k]=lo;continue;}
    BKey key={v,lo,hi};auto it=cache.find(key);
    if(it!=cache.end()){next[k]=it->second;continue;}
    int id=nodes.size();nodes.push_back({v,lo,hi});cache.emplace(key,id);next[k]=id;
   }
   cur.swap(next);count/=2;
```

```
 }
 roots[j]=cur[0];
}
// Emit gates with structural sharing, constant folding, and NOT sharing.
vector<Gate> gates;
unordered_map<GKey,int,HKey> gcache;gcache.reserve(nodes.size()*2);
vector<int> ready(n+1);int A=0;
auto mk=[&](int t,int a,int b)->int {if(t!=2&&a>b)swap(a,b); GKey k{t,a,b}; auto it=gcache.find(k);if(it!=
    gcache.end())return it->second;
 int id=n+1+(int)gates.size();gates.push_back({t==0?"AND2":t==1?"OR2":"NOT",a,b}); gcache.emplace(k,id);A
    +=(t==0?aa:t==1?ao:an);return id;};
int ONE=mk(2,0,0); // canonical 1
function<int(int)> neg=[&](int a)->int{
 if(a==0)return ONE;if(a==ONE)return 0;
 auto it=gcache.find({2,a,0});if(it!=gcache.end())return it->second;
 if(a>n){Gate &g=gates[a-(n+1)];if(g.type=="NOT")return g.a;}
 return mk(2,a,0);
};
function<int(int,int)> land=[&](int a,int b)->int{if(a==0||b==0)return 0;if(a==ONE)return b;if(b==ONE)
    return a;if(a==b)return a;if(neg(a)==b)return 0;return mk(0,a,b);};
function<int(int,int)> lor=[&](int a,int b)->int{if(a==ONE||b==ONE)return ONE;if(a==0)return b;if(b==0)
    return a;if(a==b)return a;if(neg(a)==b)return ONE;return mk(1,a,b);};
vector<int> done(nodes.size(),-1);done[0]=0;done[1]=ONE;
function<int(int)> build=[&](int id)->int {if(done[id]>=0)return done[id];auto node=nodes[id];int x=node.v
    +1;
 int lo=build(node.lo),hi=build(node.hi),result;
 if(lo==hi)result=lo;
 else if(lo==0)result=land(x,hi);
 else if(hi==0)result=land(neg(x),lo);
 else if(lo==ONE)result=lor(neg(x),hi);
 else if(hi==ONE)result=lor(x,lo);
 else result=lor(land(neg(x),lo),land(x,hi));
 done[id]=result;return result;
};
vector<int> outs;for(int r:roots)outs.push_back(build(r));
// remove unused gates including unnecessary ONE if not needed; preserve ID indexing by regenerating used
    gates.
vector<char> use(gates.size());function<void(int)> mark=[&](int id){if(id<=n)return;int j=id-n-1;if(use[j])
    return;use[j]=1;mark(gates[j].a);if(gates[j].type!="NOT")mark(gates[j].b);};
for(int o:outs)mark(o);
vector<int> new_ids(gates.size());int k=n+1;
for(size_t i=0;i<gates.size();i++)if(use[i])new_ids[i]=k++;
for(size_t i=0;i<gates.size();i++)if(use[i]){auto g=gates[i];auto fix=[&](int x){return x<=n?x:new_ids[x-n
    -1];}; out<<new_ids[i]<<" "<<g.type<<" "<<fix(g.a);if(g.type!="NOT")out<<" "<<fix(g.b);out<<"\n";}
for(int j=0;j<m;j++){if(j)out<<" ";out<<(outs[j]<=n?outs[j]:new_ids[outs[j]-n-1]);}out<<"\n";
return 0;
}
```

附录 B　辅助复现实验的配置记录

编译与运行

```
g++ -O2 -std=c++17 -o synth ufug2106_stage1_bdd_baseline.cpp
./evaluator ./synth ./stage1 --report baseline.csv
# Variant example (requires variant.cpp from experiment pack):
g++ -O2 -std=c++17 -o variant variant.cpp
ORDER=reverse ./evaluator ./variant ./stage1 --report reverse.csv
REWRITE=1 ORDER=natural ./evaluator ./variant ./stage1 --report rewrite.csv
```

实验解释范围

25 个案例的逐案 Area/Delay 及 `target_met` 来自给定 Evaluator 的报告；160-seed 搜索仅用于小型案例的离线研究，所用时间不能视为单候选 Runtime。本文所列优化策略中，真正实测的是固定顺序、反向顺序、面积优先 De Morgan 改写、局部到达时间保护、Delay-first 改写和少量随机变量顺序候选。其余策略均为尚待验证的研究方向。